权益平台接入手册

> 版本日期：2026-06-26
>
> 本文档用于接入方对接权益平台下单、鉴权、签名、回调通知与订单查询能力。本文仅覆盖当前手册涉及的接口。

文档目录

1. 1. 文档说明
    2. 1.1 适用范围
    3. 1.2 环境地址
    4. 1.3 通用约定
5. 2. 接入前准备与对接流程
    6. 2.1 接入凭证
    7. 2.2 对接流程
    8. 2.3 幂等与重试原则
9. 3. 鉴权接口
    10. 3.1 getToken 鉴权接口
        11. 3.1.1 接口说明
        12. 3.1.2 请求参数
        13. 3.1.3 请求示例
        14. 3.1.4 Curl 示例
        15. 3.1.5 响应示例
16. 4. 签名规则与示例代码
    17. 4.1 签名算法
    18. 4.2 生成步骤
    19. 4.3 分接口签名字段

20. 4.3.1 直充下单 createCouponOrder
21. 4.3.2 订单查询 getRechargeOrder
22. 4.4 待签名字符串示例
23. 4.4.1 下单接口
24. 4.4.2 查询接口
25. 4.5 语言示例代码
26. Java
27. Node.js
28. Python
29. Go
30. 4.6 验签自查清单
31. 5. 业务接口
32. 5.1 直充下单接口 createCouponOrder
33. 5.1.1 接口说明
34. 5.1.2 请求参数
35. 5.1.3 响应参数
36. 5.1.4 请求与响应示例
37. 5.1.5 错误码与处理建议
38. 5.2 回调通知说明（notify_url）
39. 5.2.1 回调机制
40. 5.2.2 回调字段
41. 5.2.3 回调示例
42. 5.3 订单查询接口 getRechargeOrder
43. 5.3.1 接口说明
44. 5.3.2 请求参数
45. 5.3.3 响应参数
46. 5.3.4 请求与响应示例
47. 5.3.5 推荐查询策略

48. 6. 错误码、状态与重试
    49. 6.1 通用错误码
    50. 6.2 订单状态枚举
    51. 6.3 重试策略

1. 文档说明

1.1 适用范围

接口	Method	Path	说明
标准 Token 获取	`POST`	`/cardapi/auth/getToken`	使用 `app_id` + `app_secret` 获取业务接口 Token。
直充下单	`POST`	`/cardapi/order/createCouponOrder`	创建直充订单，需 Token、IP 白名单和业务签名。
订单查询	`POST`	`/cardapi/order/getRechargeOrder`	按外部订单号查询订单状态；卡券和直充订单均可通过该接口查询。

本文不新增其他业务接口；如需要库存、商品信息、直充下单等能力，请以双方另行确认的接口文档为准。

1.2 环境地址

环境	base_url	说明
测试环境	`http://apitest.kalaiwang.com`	联调、测试使用。
正式环境	`https://api.kalaiwang.com`	生产调用使用；上线前需完成凭证、IP 白名单和商品配置确认。

1.3 通用约定

项目	约定
请求方式	本文接口均使用 `POST`。
Content-Type	`application/json`。
业务接口鉴权	下单、查询等业务接口需传 `Authorization: Bearer <token>`。平台兼容裸 JWT，但建议统一使用 Bearer 格式。
响应信封	统一返回 `{"code": 200, "msg": "操作成功", "data": ...}`；`code=200` 表示业务处理成功。
字段命名	接入方请求体使用 `snake_case`；平台回调字段使用 `lowerCamelCase`。
时间戳	请求中的 `timestamp` 使用毫秒级时间戳字符串，例如 `1718093632000`。
金额单位	接口金额字段按分处理，例如 `1000` 表示 10.00 元。
字符集	签名、请求体和回调均按 UTF-8 处理。

2. 接入前准备与对接流程

2.1 接入凭证

接入前请向平台运营或技术支持获取并确认以下信息：

凭证/配置	是否必需	使用位置	说明
`app_id`	标准接入必需	`getToken`	应用标识，由平台分配。
`app_secret`	标准接入必需	`getToken`	应用密钥，与 `app_id` 配套。
`company_code` / `appId`	Diy 接入必需	`getTokenDiy`	Diy Token 获取接口使用；请求字段名为 `appId` 。
`aesKey`	必需	业务签名、卡密加密、回调验签	16 位 AES 密钥。严禁泄露，不要写入前端代码或公开日志。
IP 白名单	业务接口必需	下单、查询	业务接口会校验调用方 IP；未报备会返回 `501 / 不在IP白名单` 。
商品配置	必需	下单	确认可售商品、价格、库存、限额和商品类型。

2.2 对接流程

```
1. 申请并确认 app_id/app_secret 或 company_code、aesKey、IP 白名单、商品配置
2. 调用 /cardapi/auth/getToken 获取 Token
3. 按第 4 章签名规则生成业务请求 sign
4. 携带 Authorization + 请求体 + sign 调用 /cardapi/order/createCouponOrder 下单
5. 保存平台订单号 order_sn 与外部订单号 out_order_sn 的对应关系
6. 监听 notify_url 回调；收到回调后仍调用查询接口做二次确认
7. 未收到回调或结果未知时，调用 /cardapi/order/getRechargeOrder 主动查询兜底
8. 按 order_status 的终态处理发货、成功、失败、退款或人工介入
```

2.3 幂等与重试原则

- `out_order_sn` 是接入方外部订单号，必须全局唯一。建议使用业务订单号或业务订单号加渠道前缀。
- 下单请求超时、网络断开或未收到明确响应时，不要直接换新订单号重复下单；应先使用原 `out_order_sn` 调用查询接口确认。
- 重复提交相同 `out_order_sn` 可能返回 `订单已存在，请勿重复提交` ，也可能返回原订单信息；请按查询结果做最终判断。
- 签名错误、字段缺失、凭证错误、IP 白名单、余额不足、商品类型不匹配等问题不建议自动重试，应修正配置或请求后再发起。
- Token 过期可重新获取 Token 后重试；系统繁忙、网络超时可按退避策略重试，并结合查询接口确认订单状态。

3. 鉴权接口

3.1 getToken 鉴权接口

3.1.1 接口说明

项目	说明
接口名称	标准 Token 获取
请求方式	`POST`
接口 URL	`${base_url}/cardapi/auth/getToken`
Content-Type	`application/json`
是否需要 Authorization	否
Token 有效期	默认 2 小时；平台可通过配置调整。过期后业务接口会返回 `5001 / token过期` 或 `501 / token过期` 。

3.1.2 请求参数

字段	类型	必填	说明	示例
`app_id`	String	是	平台分配的应用标识。	`your_app_id`
`app_secret`	String	是	平台分配的应用密钥。	`your_app_secret`

3.1.3 请求示例

```
POST ${base_url}/cardapi/auth/getToken
Content-Type: application/json

{
  "app_id": "your_app_id",
  "app_secret": "your_app_secret"
}
```

3.1.4 Curl 示例

```
curl -X POST "${base_url}/cardapi/auth/getToken" \
  -H "Content-Type: application/json" \
  -d '{
    "app_id": "your_app_id",
    "app_secret": "your_app_secret"
  }'
```

3.1.5 响应示例

```
{
  "code": 200,
  "msg": "操作成功",
  "data": {
    "token": "eyJhbGciOiJIUzUxMiJ9.xxxxx..."
  }
}
```

```
{
  "code": 401,
  "msg": "凭证错误"
}
```

4. 签名规则与示例代码

4.1 签名算法

项目	规则
算法	`AES/ECB/PKCS5Padding`（AES-128 ECB，PKCS5Padding）
密钥	平台分配的 16 位 `aesKey`
字符集	`UTF-8`
输出	AES 加密后的字节数组进行 Base64 编码，作为请求体中的 `sign`

项目	规则
参与签名字段	不同业务接口不同，详见 4.3；`sign` 本身不参与签名。

请勿将 `aesKey` 作为业务请求体字段发送给平台；它只参与本地生成签名。

4.2 生成步骤

步骤	操作	说明
1	组装参数	取业务请求字段，并追加本地保存的 `aesKey`。
2	排序	按 key 的 ASCII 升序排序。
3	拼接字符串	按 `key=value` 拼接，字段之间使用英文逗号 `,` 分隔，不追加尾逗号。
4	AES 加密	使用 `aesKey` 作为 AES 密钥，对待签名字符串做 `AES/ECB/PKCS5Padding` 加密。
5	Base64	将加密结果 Base64 编码，得到 `sign`。

4.3 分接口签名字段

4.3.1 直充下单 createCouponOrder

排序后字段顺序	字段来源	说明
`aesKey`	平台分配	AES 密钥，仅参与签名。
`nonce_str`	请求体	随机字符串。
`notify_url`	请求体	回调地址。
`num`	请求体	购买数量。
`out_order_sn`	请求体	外部订单号。
`product_id`	请求体	直充商品编号。
`timestamp`	请求体	毫秒级时间戳。

4.3.2 订单查询 getRechargeOrder

排序后字段顺序	字段来源	说明
`aesKey`	平台分配	AES 密钥，仅参与签名。
`nonce_str`	请求体	随机字符串。
`out_order_sn`	请求体	外部订单号。
`timestamp`	请求体	毫秒级时间戳。

4.4 待签名字符串示例

4.4.1 下单接口

```
aesKey=KRUScDwbArhbZykJ,nonce_str=rV69DA1lWW6KmnJzhcGPSBm7SvkFiM1h,notify_url=https://api.partner.com/callback/coupon,num=1,out_order_sn=PARTNER2
0240611001,product_id=42600299,timestamp=1718093632000
```

4.4.2 查询接口

```
aesKey=KRUScDwbArhbZykJ,nonce_str=aB3CD4eF5gH6iJ7kL8mN9oP0qR,out_order_sn=PARTNER20240611001,timestamp=1718093650000
```

以上字符串使用 `KRUScDwbArhbZykJ` 作为 AES 密钥加密后，再 Base64 编码，即得到请求体中的 `sign` 。

4.5 语言示例代码

Java

```
import javax.crypto.Cipher;
import javax.crypto.spec.SecretKeySpec;
import java.nio.charset.StandardCharsets;
import java.util.Base64;
import java.util.Map;
import java.util.TreeMap;
import java.util.stream.Collectors;

public class KalaiwangSignUtil {
    public static String generateSign(Map<String, Object> params, String aesKey) throws Exception {
```

```
        TreeMap<String, Object> sorted = new TreeMap<>(params);
        sorted.put("aesKey", aesKey);

        String content = sorted.entrySet().stream()
                .map(e -> e.getKey() + "=" + e.getValue())
                .collect(Collectors.joining(","));

        Cipher cipher = Cipher.getInstance("AES/ECB/PKCS5Padding");
        SecretKeySpec keySpec = new SecretKeySpec(aesKey.getBytes(StandardCharsets.UTF_8), "AES");
        cipher.init(Cipher.ENCRYPT_MODE, keySpec);
        byte[] encrypted = cipher.doFinal(content.getBytes(StandardCharsets.UTF_8));
        return Base64.getEncoder().encodeToString(encrypted);
    }
}
```

Node.js

```
const crypto = require('crypto')

function generateSign(params, aesKey) {
  const sorted = { ...params, aesKey }
  const content = Object.keys(sorted)
    .sort()
    .map(key => `${key}=${sorted[key]}`)
    .join(',')

  const cipher = crypto.createCipheriv('aes-128-ecb', Buffer.from(aesKey, 'utf8'), null)
  cipher.setAutoPadding(true)
  let encrypted = cipher.update(content, 'utf8', 'base64')
  encrypted += cipher.final('base64')
  return encrypted
}
```

Python

```
import base64
from Crypto.Cipher import AES
```

```
def generate_sign(params: dict, aes_key: str) -> str:
    data = dict(params)
    data['aesKey'] = aes_key
    content = ','.join(f'{k}={data[k]}' for k in sorted(data.keys()))

    raw = content.encode('utf-8')
    pad_len = 16 - len(raw) % 16
    raw += bytes([pad_len]) * pad_len

    cipher = AES.new(aes_key.encode('utf-8'), AES.MODE_ECB)
    return base64.b64encode(cipher.encrypt(raw)).decode('utf-8')
```

Go

```
package main

import (
    "bytes"
    "crypto/aes"
    "encoding/base64"
    "fmt"
    "sort"
    "strings"
)

func GenerateSign(params map[string]interface{}, aesKey string) (string, error) {
    params["aesKey"] = aesKey
    keys := make([]string, 0, len(params))
    for k := range params { keys = append(keys, k) }
    sort.Strings(keys)

    parts := make([]string, 0, len(keys))
    for _, k := range keys { parts = append(parts, fmt.Sprintf("%s=%v", k, params[k])) }
    content := strings.Join(parts, ",")

    block, err := aes.NewCipher([]byte(aesKey))
```

```
    if err != nil { return "", err }
    padLen := block.BlockSize() - len([]byte(content))%block.BlockSize()
    plain := append([]byte(content), bytes.Repeat([]byte{byte(padLen)}, padLen)...)

    encrypted := make([]byte, len(plain))
    for start := 0; start < len(plain); start += block.BlockSize() {
        block.Encrypt(encrypted[start:start+block.BlockSize()], plain[start:start+block.BlockSize()])
    }
    return base64.StdEncoding.EncodeToString(encrypted), nil
}
```

4.6 验签自查清单

检查项	正确做法	常见错误
字段名	严格使用文档字段名，`aesKey` 为 camelCase，其余业务字段多为 snake_case。	写成 `aes_key`、`nonceStr`、`notifyUrl`。
排序	按 key 的 ASCII 升序排序。	按请求 JSON 原始顺序或按中文说明顺序拼接。
分隔符	字段之间使用英文逗号 `,`。	使用 `&` 或末尾多一个逗号。
sign	`sign` 不参与签名。	把 `sign` 放进待签名字段。
num	保持与请求体一致，示例为数字 `1`。	签名时使用 `1.0` 或空值。
notify_url	签名使用的 URL 必须与请求体完全一致。	签名时 URL 编码、去掉斜杠或改大小写。
密钥长度	`aesKey` 必须为 16 位字符串。	密钥为空、长度不是 16 位或使用错误环境的密钥。

5. 业务接口

本章按接入调用链路，将直充下单、回调通知、订单查询合并说明。三个子章节分别对应下单发起、异步通知、主动查询兜底。

5.1 直充下单接口 createCouponOrder

5.1.1 接口说明

项目	说明
接口名称	直充下单
请求方式	`POST`

项目	说明
接口 URL	`${base_url}/cardapi/order/createCouponOrder`
Content-Type	`application/json`
鉴权方式	`Authorization: Bearer <token>` + IP 白名单 + 业务签名
接口用途	创建直充订单，扣减账户余额并进入发货流程；部分自营库存可同步返回加密卡密。

5.1.2 请求参数

5.1.2.1 请求头

Header	必填	说明
`Authorization`	是	`Bearer <token>`，Token 由鉴权接口获取。
`Content-Type`	是	固定 `application/json`。

5.1.2.2 请求体字段

字段	类型	必填	约束	说明
`product_id`	String	是	非空	直充商品编号，由平台分配。
`out_order_sn`	String	是	非空，全局唯一	接入方外部订单号。结果未知时使用该字段查询。
`num`	Long	是	建议 1-10，平台限制最大 10	购买数量。
`timestamp`	String	是	毫秒级时间戳	当前请求时间戳，例如 `1718093632000`。
`notify_url`	String	是	公网可访问	订单成功后的异步通知地址。
`nonce_str`	String	是	建议 16-32 位随机字符串	随机字符串，用于签名。
`sign`	String	是	非空	按第 4 章规则生成的签名值。

5.1.2.3 常见字段校验提示

字段/场景	错误提示
`product_id` 为空	直充编号不能为空
`out_order_sn` 为空	外部订单号不能为空

字段/场景	错误提示
`num` 为空	购买数量不能为空
`num > 10`	must be less than or equal to 10
`timestamp` 为空	时间戳不能为空
`notify_url` 为空	回调地址不能为空
`nonce_str` 为空	随机字符串不能为空
`sign` 为空	签名不能为空

5.1.3 响应参数

5.1.3.1 通用响应信封

字段	类型	说明
`code`	Integer	业务状态码，`200` 表示成功。
`msg`	String	提示信息。
`data`	Object	业务数据，失败时可能不存在。

5.1.3.2 data 字段

字段	类型	是否一定返回	说明
`order_sn`	String	是	平台订单号。
`out_order_sn`	String	是	回显接入方外部订单号。
`coupon_list`	String	否	部分自营库存即时出库时返回；内容为 AES 加密后的卡密列表 JSON，使用 `aesKey` 解密。

> `coupon_list` 不保证每笔订单同步返回。未返回时，请通过回调和查询接口确认订单最终状态。

5.1.4 请求与响应示例

5.1.4.1 HTTP 请求示例

```
POST ${base_url}/cardapi/order/createCouponOrder
Authorization: Bearer eyJhbGciOiJIUzUxMiJ9.xxxxx...
Content-Type: application/json

{
  "product_id": "42600299",
  "out_order_sn": "PARTNER20240611001",
  "num": 1,
  "timestamp": "1718093632000",
  "notify_url": "https://api.partner.com/callback/coupon",
  "nonce_str": "rV69DA1lWW6KmnJzhcGPSBm7SvkFiM1h",
  "sign": "按第4章签名规则生成的Base64字符串"
}
```

5.1.4.2 Curl 示例

```
curl -X POST "${base_url}/cardapi/order/createCouponOrder" \
  -H "Authorization: Bearer eyJhbGciOiJIUzUxMiJ9.xxxxx..." \
  -H "Content-Type: application/json" \
  -d '{
    "product_id": "42600299",
    "out_order_sn": "PARTNER20240611001",
    "num": 1,
    "timestamp": "1718093632000",
    "notify_url": "https://api.partner.com/callback/coupon",
    "nonce_str": "rV69DA1lWW6KmnJzhcGPSBm7SvkFiM1h",
    "sign": "按第4章签名规则生成的Base64字符串"
  }'
```

5.1.4.3 成功响应示例

```
{
  "code": 200,
  "msg": "操作成功",
  "data": {
    "order_sn": "2024061114302500012345",
```

```
    "out_order_sn": "PARTNER20240611001"
  }
}
```

5.1.4.4 同步返回卡密示例

```
{
  "code": 200,
  "msg": "操作成功",
  "data": {
    "order_sn": "2024061114302500012345",
    "out_order_sn": "PARTNER20240611001",
    "coupon_list": "Base64_AES_Encrypted_String"
  }
}
```

`coupon_list` 解密后的 JSON 结构示例：

```
[
  {
    "couponCode": "COUPON_CODE_001",
    "couponKey": "COUPON_KEY_001",
    "validTime": "2026-12-31"
  }
]
```

5.1.5 错误码与处理建议

code	msg	触发场景	是否建议重试	处理建议
400	字段校验提示	缺字段、字段类型错误、`num` 超限	否	修正请求体后重新提交。
501	token不可为空	未传 `Authorization`	否	先获取 Token 并放入请求头。
5001	token过期	Token 不存在、过期或被覆盖	是	重新获取 Token 后重试；若下单结果未知，先查单。
501	不在IP白名单	调用方 IP 未报备	否	联系平台添加 IP 白名单。

code	msg	触发场景	是否建议重试	处理建议
501	验签不合规!	平台计算签名与请求 `sign` 不一致	否	按第 4 章检查字段、排序、分隔符和密钥。
501	验签失败!	AES 加密或签名过程异常	否	检查 `aesKey` 是否为 16 位、环境是否匹配。
503	该产品不是直充类型!	`product_id` 对应商品不是直充	否	确认商品类型或更换正确商品。
503	商品限额，请明日购买!	超出商品限额	否	次日再下单或联系平台调整限额。
503	卡券价格异常!	价格配置异常	否	联系平台排查商品价格。
503	该商品库存不足!	商品库存不足	谨慎	稍后查询库存/联系平台；不要高频重试。
504	账户不存在!	账户状态或组织配置异常	否	联系平台核对账户。
504	账户余额不足,请充值!	余额不足	否	充值后再下单。
505	订单已存在，请勿重复提交	`out_order_sn` 重复	否	调用查询接口确认原订单状态。
505	订单生成失败	订单落库或业务处理失败	谨慎	先按原 `out_order_sn` 查询；确认不存在后再重试。
507	限流请稍后重试	限流/拦截逻辑异常或系统繁忙	是	退避后重试；避免并发刷接口。

5.2 回调通知说明（notify_url）

5.2.1 回调机制

下单时传入的 `notify_url` 用于接收平台订单成功后的异步通知。回调通知只表示平台侧状态变化，请接入方收到回调后仍调用订单查询接口做二次确认。

项目	说明
回调方式	`POST`
Content-Type	`application/json`
触发时机	订单状态达到成功状态后，由平台任务向 `notify_url` 发送。
成功确认	接入方返回 JSON，且 `code=200` 表示接收成功。
重试次数	平台会记录回调次数；未收到 `code=200` 时会继续尝试，最多按平台当前逻辑尝试至 3 次标记。
最终状态依据	以 `/cardapi/order/getRechargeOrder` 查询结果为准。

5.2.2 回调字段

字段	类型	说明
`orderSn`	String	平台订单号。
`outOrderSn`	String	接入方外部订单号。
`type`	Integer	商品类型：`0`=卡券，`1`=直充。
`totalPrice`	Long	订单金额，单位分。
`account`	String	充值账号；卡券类通常为空。
`totalSettlePrice`	Long	结算金额，单位分。
`timestamp`	Long	回调时间戳，毫秒级。
`nonceStr`	String	随机字符串。
`orderStatus`	String	订单状态，详见 6.2。
`sign`	String	回调签名。

回调验签建议：接入方使用本地保存的 `aesKey` 参与验签，不要信任报文中可能出现的任何密钥字段。回调签名与业务请求签名不同，平台对回调字段集合追加 `aesKey` 后，将 JSON 字符串使用 `AES/ECB/PKCS5Padding` 加密并 Base64 输出。

5.2.3 回调示例

5.2.3.1 平台回调请求示例

```
POST https://api.partner.com/callback/coupon
Content-Type: application/json

{
  "orderSn": "2024061114302500012345",
  "outOrderSn": "PARTNER20240611001",
  "type": 0,
  "totalPrice": 950,
  "account": "",
  "totalSettlePrice": 950,
  "timestamp": 1718093660000,
  "nonceStr": "48392017",
  "orderStatus": "12",
```

```
  "sign": "平台生成的回调签名"
}
```

5.2.3.2 接入方成功响应示例

```
{
  "code": 200,
  "msg": "ok"
}
```

5.2.3.3 回调处理建议

- 先按 `outOrderSn` 做幂等去重，避免重复回调导致重复发货或重复入账。
- 验签通过后更新本地回调接收记录。
- 立即调用 `/cardapi/order/getRechargeOrder` 查询一次，以查询结果为最终状态。
- 若本地处理失败，请不要返回 `code=200`，平台会按规则重试。

5.3 订单查询接口 getRechargeOrder

5.3.1 接口说明

项目	说明
接口名称	订单查询
请求方式	`POST`
接口 URL	`${base_url}/cardapi/order/getRechargeOrder`
Content-Type	`application/json`
鉴权方式	`Authorization: Bearer <token>` + IP 白名单 + 业务签名
接口用途	按 `out_order_sn` 查询订单详情和订单状态；结果未知、回调丢失或超时时应主动查询。

5.3.2 请求参数

5.3.2.1 请求头

Header	必填	说明
`Authorization`	是	`Bearer <token>`，Token 由鉴权接口获取。
`Content-Type`	是	固定 `application/json`。

5.3.2.2 请求体字段

字段	类型	必填	说明
`out_order_sn`	String	是	下单时传入的外部订单号。
`timestamp`	String	是	当前毫秒级时间戳。
`nonce_str`	String	是	随机字符串。
`sign`	String	是	按第 4 章查询接口字段生成。

> 查询接口签名字段：`aesKey`、`nonce_str`、`out_order_sn`、`timestamp`。

5.3.3 响应参数

5.3.3.1 通用响应信封

字段	类型	说明
`code`	Integer	业务状态码，`200` 表示成功。
`msg`	String	提示信息。
`data`	Object	订单详情，失败时可能不存在。

5.3.3.2 data 字段说明

字段	类型	说明
`create_time`	String	订单创建时间。
`product_name`	String	商品名称。
`product_value`	String/Number	商品面值，单位分。
`product_type`	String/Number	商品类型：`0`=直充，`1`=直充。

字段	类型	说明
num	String/Number	购买数量。
order_id	String	平台订单号。
out_order_id	String	接入方外部订单号。
order_price	String/Number	订单金额，单位分。
order_status	String	订单状态，详见 6.2。
parent_id	String	下单商品编号。
account	String	充值账号；直充类通常为空。
account_name	String	充值账号名称；直充类通常为空。

5.3.4 请求与响应示例

5.3.4.1 HTTP 请求示例

```
POST ${base_url}/cardapi/order/getRechargeOrder
Authorization: Bearer eyJhbGciOiJIUzUxMiJ9.xxxxx...
Content-Type: application/json

{
  "out_order_sn": "PARTNER20240611001",
  "timestamp": "1718093650000",
  "nonce_str": "aB3CD4eF5gH6iJ7kL8mN9oP0qR",
  "sign": "按第4章签名规则生成的Base64字符串"
}
```

5.3.4.2 Curl 示例

```
curl -X POST "${base_url}/cardapi/order/getRechargeOrder" \
  -H "Authorization: Bearer eyJhbGciOiJIUzUxMiJ9.xxxxx..." \
  -H "Content-Type: application/json" \
  -d '{
    "out_order_sn": "PARTNER20240611001",
    "timestamp": "1718093650000",
```

```
    "nonce_str": "aB3CD4eF5gH6iJ7kL8mN9oP0qR",
    "sign": "按第4章签名规则生成的Base64字符串"
  }'
```

5.3.4.3 成功响应示例

```
{
  "code": 200,
  "msg": "操作成功",
  "data": {
    "create_time": "2024-06-11 14:30:25",
    "product_name": "示例卡券-10元",
    "product_value": 1000,
    "product_type": 0,
    "num": 1,
    "order_id": "2024061114302500012345",
    "out_order_id": "PARTNER20240611001",
    "order_price": 950,
    "order_status": "12",
    "parent_id": "42600299",
    "account": "",
    "account_name": ""
  }
}
```

5.3.4.4 错误响应示例

```
{
  "code": 505,
  "msg": "订单不存在"
}
```

```
{
  "code": 501,
```

```
  "msg": "验签不合规！"
}
```

5.3.5 推荐查询策略

```
下单成功或下单结果未知 → 启动查询兜底

建议流程：
  ├── 立即保存 out_order_sn 与业务订单关系
  ├── 3–5 秒后首次查询
  ├── 之后按 3–5 秒间隔查询，建议最长 60 秒
  ├── 收到回调后仍查询一次，以查询结果为准
  └── 超过业务可接受时间仍非终态时，记录并人工介入
```

查询结果	处理建议
`order_status = 12`	成功，停止轮询，处理发货/到账成功逻辑。
`order_status = 11`	已退款，停止轮询，处理退款逻辑。
`order_status = 6`	失败，停止轮询，记录异常并按业务规则处理。
`order_status = 9`	取消，停止轮询。
其他状态	继续轮询或按超时策略人工介入。

卡券类订单（`product_type=0`）查询时直接返回本地订单状态；直充类订单（`product_type=1`）在部分渠道下会触发渠道状态同步修正。

6. 错误码、状态与重试

6.1 通用错误码

code	含义/常见 msg	是否建议重试	处理建议
200	操作成功	-	按 `data` 和订单状态继续处理。
400	参数错误、字段校验提示	否	修正请求字段。

code	含义/常见 msg	是否建议重试	处理建议
401	凭证错误	否	核对 `app_id/app_secret` 或 `appId` 。
501	签名错误、Token 错误、IP 白名单错误	否	按具体 `msg` 修正签名、Token 或白名单。
5001	token过期	是	重新获取 Token 后重试；下单结果未知时先查单。
502	系统异常	谨慎	稍后重试；如为鉴权接口可重新调用。
503	商品类型、库存、限额、价格异常	视情况	配置类问题不重试；库存短缺可稍后或联系平台确认。
504	账户不存在或余额不足	否	核对账户或充值。
505	订单不存在、订单已存在、订单生成失败	视情况	优先按 `out_order_sn` 查询确认，再决定是否重试。
507	系统繁忙、限流请稍后重试	是	退避重试，降低并发。

6.2 订单状态枚举

order_status	含义	是否终态	说明
`0`	新订单	否	订单已创建，等待后续处理。
`1`	处理中	否	平台正在处理。
`2`	已支付/待发货	否	扣款成功，等待发货或渠道处理。
`6`	失败	是	订单处理失败。
`9`	取消	是	订单已取消。
`11`	已退款	是	订单已退款。
`12`	成功	是	卡券发放/充值成功。
`13`	成功待回调/等待回调	否	渠道成功，平台等待或准备回调处理。
`18`	存疑	否	需人工核实。

6.3 重试策略

场景	建议
获取 Token 失败	凭证错误不重试；系统异常可稍后重试。

场景	建议
Token 过期	重新获取 Token 后重试业务请求。
下单超时或网络异常	先用原 `out_order_sn` 查询；确认不存在后再按原单号或双方约定策略重试。
签名失败	不自动重试；按第 4.6 节自查并修正。
库存不足/商品限额	不高频重试；可稍后查询或联系平台。
系统繁忙/限流	建议指数退避，例如 3 秒、5 秒、10 秒；避免并发重放。
回调未收到	不要等待回调作为唯一依据；按第 5.3.5 节主动查询。
查询长时间非终态	记录订单号、请求时间、响应内容，联系平台人工核实。